Graphics and Image Processing
W. Newman Editor

The Synthesis of Solids Bounded by Many Faces

I.C. Braid
University of Cambridge

A technique is presented which allows a class of solid objects to be synthesized and stored using a computer. Synthesis begins with primitive solids like a cube, wedge, or cylinder. Any solid can be moved, scaled, or rotated. Solids may also be added together or subtracted. Two algorithms to perform addition are described. For practical designers, the technique has the advantage that operations are concise, readily composed, and are given in terms of easily imagined solids. Quite short sequences of operations suffice to build up complex solids bounded by many faces.

Key Words and Phrases: computational geometry, computer-aided design, graphics, machined components, polyhedra, shape synthesis, three-dimensional modeling

CR Categories: 3.2, 8.2

1. Introduction

Many common solid objects have shapes characterized by surfaces which are composed of large numbers of simple faces. In mechanical engineering, assemblies and machined components provide examples of the class. They may have hundreds of faces, but most faces will be planar, cylindrical, or some other elementary surface form.

Copyright © 1975, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted provided that ACM's copyright notice is given and that reference is made to the publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

Author's address: Computer Laboratory, Corn Exchange Street, Cambridge, England.

A convenient way to describe these objects to a computer is by adding together or subtracting simpler primitive objects such as cubes or cylinders. This has two advantages. First, the quantity of input is less than if faces, edges, and their interconnections were input separately. Second, practical designers prefer to think of objects as volumes of material or material enclosing a given void rather than as faces and edges.

For this approach to work, efficient algorithms for adding and subtracting objects must be devised. Two such algorithms are presented in this paper. One of these (type-1 addition) combines objects that touch but do not intersect; the other (type-2 addition) combines intersecting objects.

The class of objects is restricted to those bounded by planar or cylindrical surfaces. This is done for the sake of efficiency and simplicity of the algorithms, and because a restricted set appears to take care of many applications. (See Figure 1.)

2. Definitions

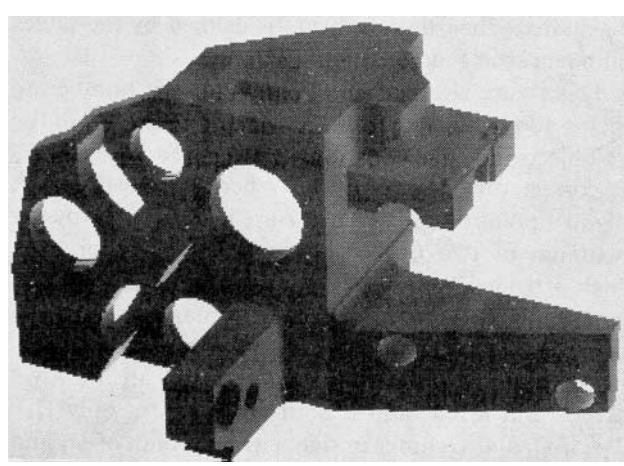
This section defines an object and the two operations of negation and addition. It explains how an object is represented and how the representation is modified when an object is transformed or made negative.

2.1 Definition of an Object

Consider, in R^3, a set of points which form a finite number of disjoint regions (a collection of regions), each region being compact, contiguous, and bounded by a piecewise differentiable surface. Each of the surface pieces with its boundary is termed a face. Two faces may intersect in one or more isolated points or in segments of a curve. Each curve segment is called an edge. An isolated point at which two faces touch, or a point in which two or more edges meet, is termed a vertex.

At each point within a face, two unit vectors normal to the surface exist, one pointing into the region, the other pointing out of the region. For any collection of

Fig. 1. A synthesized machined component.



regions, the unit vectors are chosen to be consistently inward or outward for all faces, and are called normals.

A positive object is a mapping Q from three-dimensional Euclidean space onto the set $(0, \frac{1}{2}, 1)$, defined for a collection of regions by:

$Q(q) = 1$ if q is a point within any region.
$Q(q) = \frac{1}{2}$ if q is a point in the surface of a region.
$Q(q) = 0$ otherwise, i.e. q lies outside all regions.

The points for which $Q(q) = 1$ form the *interior* of the object. The normal at a point q on the surface of a region is written $\hat{\mathbf{n}}(q, Q)$. For a positive object (sometimes written Q_p), normals point *outward.*

A negative object is a mapping Q' from three-dimensional Euclidean space onto the set $(0, -\frac{1}{2}, -1)$, defined for a collection of regions by:

$Q'(q) = -1$ if q is a point within any region.
$Q'(q) = -\frac{1}{2}$ if q is a point on the surface of a region.
$Q'(q) = 0$ otherwise, i.e. q lies outside all regions.

For a negative object (sometimes written Q_n), normals point *inward.*

2.2 Negation

A unary operator "$-$" is defined for an object Q by $Q' = -Q$ where $Q'(q) = -Q(q)$. When an object is negated, the sense of the normals at its surface is reversed, i.e. $\hat{\mathbf{n}}(Q, Q') = -\hat{\mathbf{n}}(q, Q)$.

Note that a negative object has the *same* points for its interior and its boundary as the corresponding positive object. Thus we have avoided the problems of closure encountered if an object is taken as a simple point set and its negative as the complement of the set. A negative object is probably best regarded as an object made of "antimatter" which combines with objects made of matter to produce nothing.

2.3 Addition

To allow for addition and subtraction of objects, a binary operator "$+$" is needed. Let $(Q_p, Q_n) = Q_1 + Q_2$, where Q_p and Q_n are the positive and negative objects which result from the addition, and Q_1 and Q_2 are any two objects, either of which can be positive or negative. Then the operation is defined by the tables and remarks in Figure 2.

This figure shows the advantage of the numbering scheme adopted in the definition of an object. When the two objects touch over a finite area, simple addition gives the correct values for b and c. Where the objects meet only in a point or a line, b and c are instead given by the maximum of two quantities. In fuzzy set theory [1], which provided the starting point for the present treatment, a similar rule based on the maximum of two quantities is used to specify union of two sets.

Figure 2 shows that if both Q_1 and Q_2 are positive, only Q_p will result; and if both are negative, only Q_n. Also, if Q_1 and Q_2 differ in sign, either or both of Q_p and Q_n may occur. Q_p or Q_n may each consist of several distinct pieces. If both Q_p and Q_n result, they will have boundary points in common though their interiors will be disjoint. Figure 3 illustrates addition in four different cases.

Fig. 2. Table of addition for objects.

$Q_p(q)$

$Q_2(q)$ \ $Q_1(q)$	1	$\frac{1}{2}$	0	$-\frac{1}{2}$	-1
1	1	1	1	$\frac{1}{2}$	0
$\frac{1}{2}$	1	c	$\frac{1}{2}$	b	0
0	1	$\frac{1}{2}$	0	0	0
$-\frac{1}{2}$	$\frac{1}{2}$	b	0	0	0
-1	0	0	0	0	0

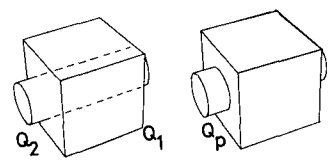
$Q_n(q)$

$Q_2(q)$ \ $Q_1(q)$	1	$\frac{1}{2}$	0	$-\frac{1}{2}$	-1
1	0	0	0	0	0
$\frac{1}{2}$	0	0	0	$-b$	$-\frac{1}{2}$
0	0	0	0	$-\frac{1}{2}$	-1
$-\frac{1}{2}$	0	$-b$	$-\frac{1}{2}$	$-c$	-1
-1	0	$-\frac{1}{2}$	-1	-1	-1

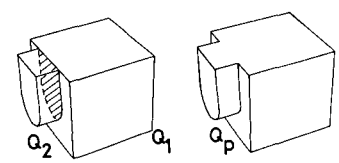
Note. If $\hat{\mathbf{n}}(q, Q_1) = -\hat{\mathbf{n}}(q, Q_2)$ and there exists a neighborhood of points about q at which the same condition holds, $b = 0$ and $c = 1$, i.e. $b = c = |\, Q_1(q) + Q_2(q) \,|$; otherwise $b = c = \frac{1}{2}$, i.e. $b = c = |\, \max(Q_1(q), Q_2(q)) \,|$.

Fig. 3. Addition of objects.

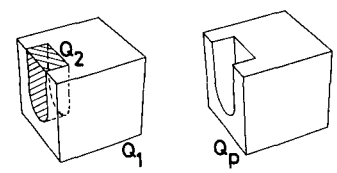
(i) Q_1, Q_2 positive and overlapping; result Q_p.



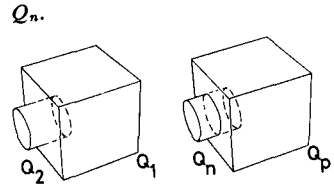
ii) Q_1, Q_2 positive and touching; result Q_p.



(iii) Q_1 positive, Q_2 negative and within or touching Q_1; result Q_p.



(iv) Q_1 positive, Q_2 negative and overlapping; result Q_p and Q_n.



2.4 Representation of Objects

Although the definition of an object as a mapping is useful when classifying types of addition, it does not suggest how an object should be modelled in a computer store. In fact, an object will be represented by a description of its surface as a collection of faces. (The letter Q

will continue to be used for objects since whether it stands for the stored object or for an object defined as a mapping (section 2.1) will be clear from its context.)

For a stored object Q, we have $Q = (V, F)$ where V is a set of vertices and F a set of faces. A face $f \in F$ is given by $f = (\mathcal{S}, B)$ where $\mathcal{S}$ is the surface in which the face lies, and B is the boundary of the face. $B = \{L\}$ where $\{L\}$ is a nonempty set of loops (cycles). A loop $L = (e_1, e_2, \ldots, e_n)$ is a sequence of edges where an edge $e_i = (\mathcal{C}_i, v_i, v_{i+1})$. $\mathcal{C}_i$ is the curve in which the edge lies and $v_i, v_{i+1} \in V$ are the vertices at the beginning and end of the edge respectively. A loop closes ($e_1 \equiv e_{n+1}$) but does not cross itself. A vertex is represented by its coordinates (x, y, z). The loop of vertices forming the outer boundary of a face is stored in an anti-clockwise sequence so that, looking from outside the face, a right-hand screw rule is followed with respect to the normals. This loop is always present and is called the *perimeter* loop of the face. Other loops, if present, define holes in the face. They are stored in clockwise sequence and are called hole loops. Objects may not have more than two faces meeting in an edge. Two faces which meet in an edge are said to be neighboring faces.

2.5 A Class of Objects

The algorithms described in this paper restrict object faces to lie in surfaces which are either planes or circular cylinders, and edges to lie in curves formed by the intersection of planes with planes or planes with cylinders. Cylinders may be circular or elliptical. Curves and surfaces are stored in the following manner.

If surface $\mathcal{S}$ is plane, it is stored as (A, B, C, D) where

$$[x\ y\ z\ 1]\begin{bmatrix} A \\ B \\ C \\ D \end{bmatrix} = 0.$$

A, B, C, and D are normalized so that $A^2 + B^2 + C^2 = 1$. The unit normal is $\hat{\mathbf{n}} = A\mathbf{i} + B\mathbf{j} + C\mathbf{k}$.

If $\mathcal{S}$ is curved, it is stored as two matrices K and L where

$$[x\ y\ z\ 1] = \left[\frac{t^2}{D(t)}\ \frac{t}{D(t)}\ \frac{1}{D(t)}\ 1\right]K + sL \qquad (1)$$

and $D(t) = (2 - \sqrt{2})t^2 + (\sqrt{2} - 2)t + 1$ and $s, t \in [0, 1]$. For the unit cylinder $x^2 + y^2 = 1$, $0 \le x, y \le 1$, $-1 \le z \le 1$,

$$K = \begin{bmatrix} 1-\sqrt{2} & 1-\sqrt{2} & 0 & 0 \\ \sqrt{2}-2 & \sqrt{2} & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} L = [0\ 0\ -2\ 0].$$

A parametric form was chosen to make display of curved edges easy. A simple exact parametric expression for a circle is a rational quadratic. The form chosen gives a more nearly linear variation in arc length with t than does the more familiar form $x = (1 - t^2)/(1 + t^2)$, $y = 2t/(1 + t^2)$. To represent a whole circle without infinite values of t, the circle must be split into several pieces and each must be separately parameterized. Four pieces have been used, the parameter t varying from 0 to 1 in each piece. The second parameter s, included to extend the unit circle to a cylindrical surface, varies from 0 to 1 causing (x, y, z) to move along a generator of the cylinder. The normal at a point $(x, y, z) = f(s, t)$ on the cylinder is given by $\mathbf{n} = \overline{\partial f}/\partial s \times \overline{\partial f}/\partial t$.

If the curve $\mathcal{C}$ in which an edge lies is straight, it is represented by storing the end points of the edge (x_1, y_1, z_1), (x_2, y_2, z_2) where

$$[x\ y\ z\ 1] = [1-t\ \ t]\begin{bmatrix} x_1 & y_1 & z_1 & 1 \\ x_2 & y_2 & z_2 & 1 \end{bmatrix}.$$

If $\mathcal{C}$ is curved, i.e. it is the intersection of a flat face and a cylindrical face, it is stored as follows. Let

$$M = \begin{bmatrix} A \\ B \\ C \\ D \end{bmatrix}$$

be the flat face and let the curved face be given by eq. (1). Then $[x\ y\ z\ 1]M = 0$ implies

$$\left[\frac{t^2}{D(t)}\ \frac{t}{D(t)}\ \frac{1}{D(t)}\ 1\right]KM + sLM = 0$$

or, rearranging,

$$s = \left[\frac{t^2}{D(t)}\ \frac{t}{D(t)}\ \frac{1}{D(t)}\ 1\right]KMN$$

where N, a scalar, is given by $N = -1/LM$. A curved edge can then be stored by keeping the matrix KMN and the interval $[t_1, t_2]$ over which t varies along the edge.

2.6 Transformation of an Object

A designer will wish to modify an object by moving, scaling, or rotating it. The modification can be expressed (see [5]) as a single transformation matrix T which acts on the object Q to produce a modified object Q'. Thus $Q' = \text{mod}\ (Q, T)$ where T is a 4×4 non-singular matrix in which the last column is

$$\begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

Every point $(x\ y\ z)$ in the object is altered to $(x'\ y'\ z')$ where $[x'\ y'\ z'\ 1] = [x\ y\ z\ 1]T$. If $Q' = (V', F')$, then $V' = \{v'\}$ where $v' = vT$, and the change in the faces is given by $F' = (\mathcal{S}', B')$. A flat face $\mathcal{S} = (A, B, C, D)$ becomes $\mathcal{S}' = (A', B', C', D')$ where

$$\begin{bmatrix} A' \\ B' \\ C' \\ D' \end{bmatrix} = T^{-1}\begin{bmatrix} A \\ B \\ C \\ D \end{bmatrix}.$$

A cylindrical face, $\mathcal{S} = (K, L)$ becomes $\mathcal{S}' = (K', L')$ where $K' = KT$ and $L' = LT$.

In general, edge equations $\{\mathcal{C}_i\}$ change when an object is transformed. However, since straight edges are given in terms of their end-points and curved edges in terms of curved face equations (both of which are altered), no change need be made to the edge equations.

For the sense of surface normals to remain unchanged, $\det(T) > 0$. If $\det(T) < 0$, a further adjustment must be made to the object description. Each loop must be reversed, i.e. $B' = \{L'\}$ where, if $L = (e_1, e_2, \ldots, e_n)$, $L' = (e'_n, e'_{n-1}, \ldots, e'_2, e'_1)$. Each edge is itself reversed in direction so that $e_i = (\mathcal{C}_i, v_i, v_{i+1})$ becomes $e'_i = (\mathcal{C}'_i, v_{i+1}, v_i)$. In practice, each edge occurs once in the forward, and once in the reverse direction, and a *reference* to an edge includes a bit that is set to indicate the reverse direction. Thus the reversal of the edge is effected by complementing this bit in each reference; the edge itself is unchanged.

Negative values of $\det(T)$ are not uncommon in practical design: a mirror image of an object is obtained by scaling in one direction by a negative scale factor, e.g.

$$T = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

2.7 Negation of an Object

When an object is made negative, each point of the object remains unchanged in space, but the sense of surface normals, the sequence within edge loops, and the sense of edges are reversed. Thus if $-Q = (V, F')$ and $F' = (\mathcal{S}', B')$, then, for a flat face, $\mathcal{S} = (A, B, C, D)$ becomes $\mathcal{S}' = (-A, -B, -C, -D)$ and thus $\hat{\mathbf{n}}' = -\hat{\mathbf{n}}$. For a cylindrical face $\mathcal{S} = (K, L)$ becomes $\mathcal{S}' = (K', L')$ where $L' = -L$ and

$$K' = K + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} L.$$

The change in K and L follows from substituting $1 - s$ for parameter s in the face equation, thereby reversing the sense of the normal $\mathbf{n}$ at each point in the face.

Since curved edges are given in terms of the equations of the faces intersecting at the edge, similar changes must be made to the edges. In fact, the term KMN (See section 2.5) becomes $K'M'N'$ where K' is as above, $M' = -M$ and $N' = N$. The sense of edges and the sequence of edge loops are reversed as described in Section 2.6 for the case when $\det(T) < 0$.

3. Addition Algorithms

3.1 Type-1 addition

Type-1 addition is the addition of two objects placed so that they meet at juxtaposed flat faces. The objects are fused where they touch at these faces, that is, at points where $b = 0$ and $c = 1$ in Figure 2. Figure 3 (ii and iii) gives examples of type-1 addition with faces cross-hatched where fusion takes place.

Fig. 4. Type-1 additions.
Note: b and c are as defined in Figure 2.

(i) Q_1, Q_2 positive; result positive.

$Q_2(q)$ \ $Q_1(q)$	1	$\frac{1}{2}$	0	$-\frac{1}{2}$	-1
1			1		
$\frac{1}{2}$			c	$\frac{1}{2}$	
0	1	$\frac{1}{2}$	0		
$-\frac{1}{2}$					
-1					

$Q_p(q)$

(ii) Q_1 positive, Q_2 negative; result positive.

$Q_2(q)$ \ $Q_1(q)$	1	$\frac{1}{2}$	0	$-\frac{1}{2}$	-1
1					
$\frac{1}{2}$					
0	1	$\frac{1}{2}$	0		
$-\frac{1}{2}$	$\frac{1}{2}$	b			
-1	0				

$Q_p(q)$

(iii) Q_1 negative, Q_2 positive; result positive.

$Q_2(q)$ \ $Q_1(q)$	1	$\frac{1}{2}$	0	$-\frac{1}{2}$	-1
1		1	$\frac{1}{2}$		0
$\frac{1}{2}$		$\frac{1}{2}$	b		
0		0			
$-\frac{1}{2}$					
-1					

$Q_p(q)$

(iv) Q_1 positive, Q_2 negative; result negative.

$Q_2(q)$ \ $Q_1(q)$	1	$\frac{1}{2}$	0	$-\frac{1}{2}$	-1
1					
$\frac{1}{2}$					
0			0		
$-\frac{1}{2}$			$-b$	$-\frac{1}{2}$	
-1		0	$-\frac{1}{2}$	-1	

$Q_n(q)$

(v) Q_1 negative, Q_2 positive; result negative.

$Q_2(q)$ \ $Q_1(q)$	1	$\frac{1}{2}$	0	$-\frac{1}{2}$	-1
1					0
$\frac{1}{2}$				$-b$	$-\frac{1}{2}$
0			0	$-\frac{1}{2}$	-1
$-\frac{1}{2}$					
-1					

$Q_n(q)$

(vi) Q_1, Q_2 negative; result negative.

$Q_2(q)$ \ $Q_1(q)$	1	$\frac{1}{2}$	0	$-\frac{1}{2}$	-1
1					
$\frac{1}{2}$					
0			0	$-\frac{1}{2}$	-1
$-\frac{1}{2}$				$-\frac{1}{2}$	$-c$
-1				-1	

$Q_n(q)$

The merit of this form of addition is that once a pair of touching faces has been found, the fusion operation is local to these faces and their neighboring faces. Furthermore, the edges of the resulting object are all edges, or pieces of edges, of the constituent objects. If a face of one object should cut a face of the other, thereby implying the creation of a new edge, the resulting object will not be defined under type-1 addition. The insistence that faces may touch but may not cut one another ensures that a single positive or negative resulting object is produced.

Considered as an operation on sets of points, type-1 addition involves all the entries in the tables of Figure 2 except those where $|Q_1(q) + Q_2(q)| > 1$. Since Q_1 and Q_2 can be positive or negative, and when they differ in sign, the result can also be positive or negative, six varieties of type-1 addition are possible: these are shown in Figure 4 of which Figure 4 (i) and (ii) match Figure 3 (ii) and (iii).

In 2.1, objects were defined as mappings onto the sets $(0, \frac{1}{2}, 1)$, $(0, -\frac{1}{2}, -1)$. Similarly, in type-1 addition, the two touching flat faces f_1 and f_2 are regarded as mappings defined over the surfaces on which they lie, onto the sets $(0, \frac{1}{2}, 1)$, $(0, -\frac{1}{2}, -1)$. The set $(0, \frac{1}{2}, 1)$ is arbitrarily assigned to f_1. Thus $f_1(q) = 1$ if q is a point

Fig. 5. Addition of two faces.

$f_2(q)$ \ $f_1(q)$	1	$\frac{1}{2}$	0
-1	0	0	0
$-\frac{1}{2}$	$\frac{1}{2}$	d	0
0	1	$\frac{1}{2}$	0

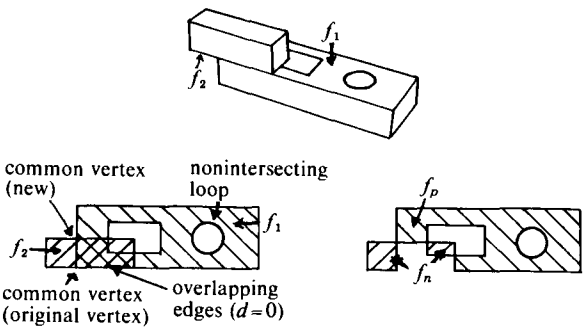
$f_p(q)$

$f_2(q)$ \ $f_1(q)$	1	$\frac{1}{2}$	0
-1	0	$-\frac{1}{2}$	-1
$-\frac{1}{2}$	0	$-d$	$-\frac{1}{2}$
0	0	0	0

$f_n(q)$

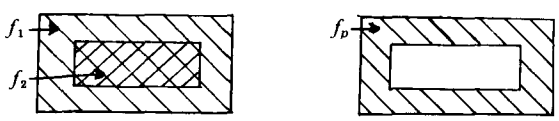
Note: $d = 0$ if the tangents to the boundary curves of f_1 and f_2 exist at q, and are equal in direction and opposite in sense, and there exists an open neighborhood of points about q at which the same condition holds; $d = \frac{1}{2}$ otherwise.

(i) Both perimeter loops contain common vertices.



(ii) One perimeter loop has common vertices.

(iii) Neither perimeter loop has common vertices.



within the face, $f_1(q) = \frac{1}{2}$ if q is a point on the face boundary, and $f_1(q) = 0$ if q lies outside the face. For f_2, $f_2(q) = -1$ for points within it, $f_2(q) = -\frac{1}{2}$ for boundary points, and $f_2(q) = 0$ for points outside the face.

After fusion, usually several faces will replace f_1 and f_2. Those with the same sense of normal as f_1 are termed positive and written as f_p; those of opposite sense are termed negative and written as f_n. Sometimes, however, type-1 addition leaves faces of only one orientation, or no faces at all. Figure 5 gives the tables for fusing two faces, and shows three examples.

Since the resulting faces are defined by their edges, an algorithm to carry out the fusion of two faces must discover points at which $|f(q)| = \frac{1}{2}$. When $f = d$ (Figure 5), it is easy to determine whether $d = 0$ or $d = \frac{1}{2}$, since this depends only on the tangents local to q. At other times a test must be made to determine whether a point on the boundary of one face is inside or outside the other face. To do this a semi-infinite ray lying in the surface of the face is drawn from the point, and the intersections of the ray and the edges of the face are counted. If the number is odd, the point lies inside the face.

The algorithm has two basic steps. In the first, each edge of f_1 is compared with each edge of f_2. Where two edges are found to cross at a point part way along each edge, the edges are split into *pieces*, and a new vertex is inserted. The vertex is marked as a *common* vertex, i.e. it is common to f_1 and f_2. If the edges cross at an original vertex or overlap, the original vertices involved are also marked as common.

In the second step, the edges and pieces of edges are linked together in new sequences to form new faces f_p or f_n. For intersecting loops, the linking is simply a threading process in which a loop is made by adding edges or pieces of edges, swapping from f_1 to f_2 or vice versa every time a common vertex is met, until the loop closes (see Figure 5). In effect the loops themselves are used instead of semi-infinite rays to determine the positions of points relative to loops. For nonintersecting loops, however, a ray must be constructed. Where possible, the algorithm deals with intersecting loops before tackling the computationally more expensive nonintersecting loops. New loops may be either perimeters or holes, and holes must be assigned to their enclosing new perimeters with further recourse to ray tests. Finally each new perimeter with its holes is stored as a new face of type f_p or f_n.

Two devices speed up the algorithm. The first is to replace a face by an enclosing rectangle and thus hasten tests of intersections with the face. It is used in the first basic step when intersections between one face and edges of the other are sought. Again, in the second step, the point in question is checked against the rectangle enclosing the face, and only if the point is not outside is a ray test performed.

The second device exploits the fact that the area of a perimeter loop is greater than the sum of the areas of its holes. When a hole is considered for assignment to a new face, its area is first compared with the area of the perimeter loop less any holes already assigned: if it is greater, the loop cannot belong to the face and a ray test can be avoided.

The sign of the area is also utilized: together with the knowledge that the loop is a perimeter or hole, it determines whether the loop belongs to a face of type f_p or f_n.

One further remark should be made. In the first step, the intersection of two finite edges must be characterized so that the appropriate action of inserting vertices, splitting edges or marking old vertices as common can be taken. To do this, the position of an intersection point with respect to each of the two edges is found. If, for one of the edges, the point lies on the finite edge, there are three possibilities: the point may be at the beginning vertex, at the end vertex, or within the finite edge. The same is true for the point with respect to the other edge, which makes a total of nine possible configurations for an intersection between two finite edges. In the implementation, the configurations are represented by num-

ber pairs (i, j), $i, j = 1, 2, 3$. This is an example of a mapping like that used in Section 2 where numbers represent membership of a set, only here the set is one-dimensional and defined on a line.

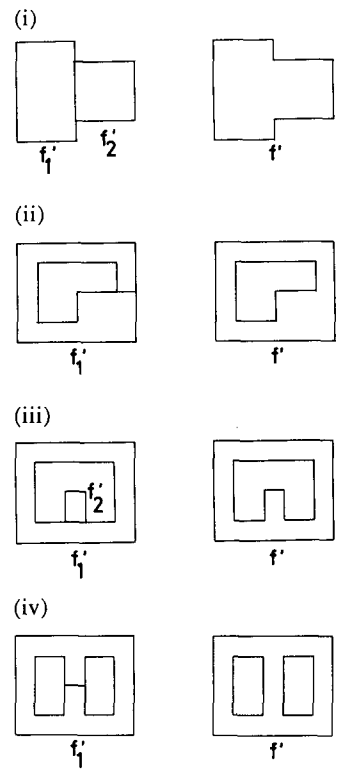
Meeting Faces. Step 1 of the algorithm contains a procedure for meeting faces, entered on occasions when $d = 0$ (see Figure 5). The faces which meet at this part of the boundary of the faces being fused are examined: if they meet without change of slope they can be made into a single face. Sometimes the meeting faces are in fact a single face which afterwards becomes a face with an additional hole. Figure 6 gives a table and examples of the addition of two meeting faces, f_1' and f_2'. Since they meet without change of slope, both faces must have the same normal, and both are represented by mappings f' onto $(0, \frac{1}{2}, 1)$.

The algorithm for adding meeting faces distinguishes the four cases shown in Figure 6 on the basis of whether two faces are involved (i and iii) or one face (ii and iv), and whether the edge where $e = 1$ is in a perimeter or hole. In case (ii) the areas of the loops are used to decide which of the two loops surrounds the other and must therefore be stored as a perimeter.

Fig. 6. Addition of meeting faces.

		$f_1'(q)$		
		1	$\frac{1}{2}$	0
$f_2'(q)$	1			1
	$\frac{1}{2}$		e	$\frac{1}{2}$
	0	1	$\frac{1}{2}$	0
		$f'(q)$		

Note: $e = 1$ if the tangents to the boundary curves exist at q, and if they are equal in direction and opposite in sense, and there exists an open neighborhood of points about q at which the same condition holds; $e = \frac{1}{2}$ otherwise.



3.2 Type-2 Addition

The second type of addition permits objects to intersect. One object (Q_1) may be any positive or negative object; the other (Q_2) must be a negative cube or a cylinder, either transformed or not. The two objects must be positioned so that no curved face of Q_1 intersects a curved face of Q_2. If Q_1 is positive, the resulting positive object Q_p is retained and Q_n is discarded. If Q_1 is negative, no Q_p can occur, and the result is Q_n. Figure 7 gives the table entries for type-2 addition, and shows objects before and after addition. Notice (Figure 7 (ii)) that two negative objects may overlap ($Q_1(q) + Q_2(q) < 1$). Type-2 addition was first developed with drilling and cutting operations in mind. The negative addition of Figure 7 (ii) was a happy accident.

Unlike type-1 addition, where fusion was largely confined to a pair of coplanar faces, type-2 addition affects potentially every edge and face of Q_1 and Q_2. Q_2 is simple and convex, however, and the effects of addition are local to it: any face of Q_1 which does not cut or touch Q_2 will be unaffected by the operation. To determine which these faces are, both objects are first transformed to bring Q_2 back to its untransformed state at the origin. It can then be enclosed by a sphere, and the surface in which each face of Q_1 lies can easily be tested to see if it cuts the sphere. If not, the face and all its edges need not be considered further.

Next the remaining faces and edges of Q_1 and all the faces and edges of Q_2 must be compared, an edge from one object with a face of the other. As in type-1 addition, a numbering scheme is used to denote the position of an intersection point with respect to the finite edge and finite face. Only if the point lies on the finite edge is a ray test begun to find where the point is with respect to the face. A number is assigned according to whether it is inside the face, outside, or on the face boundary. The edge is split if necessary and the intersection with the face recorded in a list by saving the vertex, edge and face number together with a reference to the part of the edge interior to the face. If this part is of zero length or if the edge lies in the face, the intersection can be ignored. The vertex is marked as a common vertex, i.e. common to Q_1 and Q_2.

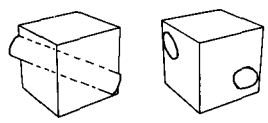
The second stage of addition joins up the pieces to form new faces which in turn form the final object. Where faces intersect, new edges must be constructed. These always begin and end at common vertices.

Type-2 addition often requires a test to determine whether a point lies inside an object or not. This is easily determined for Q_2, but for Q_1 a ray test is needed. Fortunately, except in the case where Q_1 totally surrounds but does not intersect Q_2, this test can be avoided. By making use of local properties of connectivity and the sense of normals and edges, new faces can be built up by stepping along the intersections of Q_1 and Q_2. This is the three-dimensional equivalent of the threading process used to link edges in type-1 addition. Intersecting loops are again processed before noninter-

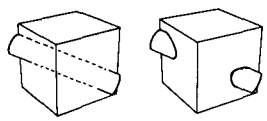
Fig. 7. Type-2 addition.
Note: b and c are as defined in Figure 2.

$$
\begin{array}{c|ccc}
 & & Q_1(q) & \\
 & 1 & \frac{1}{2} \quad 0 \quad -\frac{1}{2} & -1 \\
\hline
1 & & & \\
\frac{1}{2} & & & \\
Q_2(q) \quad 0 & 1 & \frac{1}{2} \quad 0 & \\
-\frac{1}{2} & \frac{1}{2} & b \quad 0 & \\
-1 & 0 & 0 \quad 0 & \\
 & & Q_p(q) &
\end{array}
\qquad
\begin{array}{c|ccc}
 & & Q_1(q) & \\
 & 1 & \frac{1}{2} \quad 0 \quad -\frac{1}{2} & -1 \\
\hline
1 & & & \\
\frac{1}{2} & & & \\
Q_2(q) \quad 0 & & 0 \quad -\frac{1}{2} & -1 \\
-\frac{1}{2} & & -\frac{1}{2} \quad -c & -1 \\
-1 & & -1 \quad -1 & -1 \\
 & & Q_n(q) &
\end{array}
$$

(i) Q_1 positive, Q_2 negative; result Q_p.



(ii) Q_1, Q_2 negative; result Q_n.



secting loops, thereby reducing the number of ray tests made for faces. The area test is particularly valuable in the assignment of holes to perimeters, a more arduous task than in Type-1 addition as there can now be perimeters within holes within perimeters, and so on.

Sometimes fast logical computations can replace slower geometric tests. In the final stage of the algorithm, nonintersecting faces of Q_2 are added to the resulting object until the condition that every edge should meet two faces holds throughout the object. A bit map with one bit per edge is used to find edges where the condition is not satisfied, and appropriate faces from Q_2 are added. The process is then repeated once more as the newly-added faces may introduce further edges which meet only one face. Because of the simple form of Q_2, two passes are always sufficient.

Type-2 addition assumes Q_1 is negative and always discards parts of Q_1 *inside* the cube or cylinder forming Q_2. It is possible to allow positive Q_2 provided the expected result is clearly defined. Thus for addition of positive Q_1 and Q_2, parts of Q_1 inside Q_2 will still be discarded, but the tests based on local orientations of edges and faces, which occur at various points in the algorithm, must be reversed. For positive Q_2 and negative Q_1, if a positive result is desired, it is necessary to discard parts of Q_1 inside Q_2 and reverse the local tests. Probably the best solution is to make Q_1 and the object resulting from type-2 addition always positive. The user will then make Q_2 negative, if he wants to use it as a drill or saw, and make Q_2 positive to achieve an addition. He will be unaware that internally the addition is carried out by making Q_1 and Q_2 negative, performing the addition and then negating the result.

Finally, a comment must be made about degenerate cases—a real hazard in computational geometry. The system designer has three options: he can warn the user that if he sets up certain degenerate cases disaster will ensue, or the program can be made to recognize the cases and to alert the user, or the program can be extended to handle these cases correctly. It is worthwhile distinguishing reasonable from what might be termed malicious degenerate cases, e.g. adding an object to its exact negative to produce nothing.

In type-2 addition, the algorithm handles the case where one or more faces of Q_1 are coplanar with a face of Q_2. If the faces of Q_1 have the same normal as the face of Q_2, they tend to expand, and possibly to coalesce, by addition of parts of the face of Q_2. When the normals are equal and opposite, the coplanar faces tend to contract, and sometimes to split into smaller, distinct faces or to disappear. This case is similar to type-1 addition, but not quite as straightforward as the assumption that the faces which are neighbors of the coplanar faces do not intersect no longer holds.

A second type of degeneracy is not handled. It occurs when one object touches another but only in an edge or at a vertex. This was felt not to be useful in real design; furthermore it could lead to objects which violate the early assumption that only two faces meet in any edge.

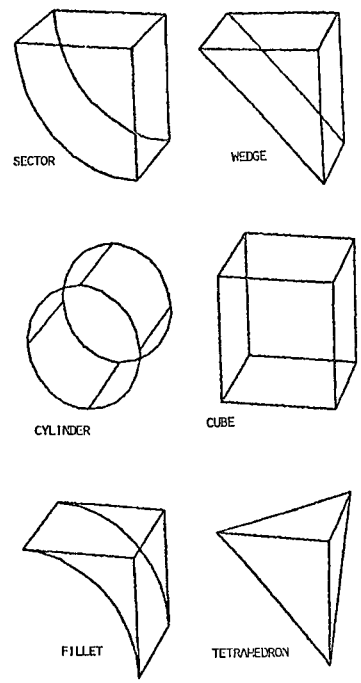
3.3 Implementation

An implementation of these algorithms was written for the Titan computer in Fortran and SAL [4]. The program accepts typed commands and builds up a stored description of each shape. Wire-frame pictures can be sent to a PDP7 computer and refresh display, to a storage tube display, or to a plotter. The stored description can be further processed to produce shaded pictures (e.g. Figure 1) or pictures with hidden-lines removed, or to calculate volume, centroid, moments of inertia, and principal axes of objects. A second version of the program has been developed as a set of subroutines to be called from a Fortran program. It is useful for designing *families* of shapes, each family being represented by a subroutine whose arguments are the dimensions of a member of the family.

The primitives are a cube, wedge, tetrahedron, cylinder, sector, and fillet. The designer can also make a generalized primitive by giving a sequence of straight lines and axes in the xy-plane. From this sequence, an object of constant thickness and similar profile is constructed by the program. The designer can move, rotate or scale objects using the MODIFY command. COMBINE causes type-1 addition to occur at all coplanar faces of opposite sense, and INTERSECT initiates a single type-2 addition between a given object and primitive cube or cylinder. (See Figure 8.)

The program runs in 12K of 48 bit words, of which 3K are reserved for object descriptions, $3\frac{1}{2}$K for the root and 5K for a single overlay segment. For complex objects like that in Figure 1, the data space is increased by running the program in 16K. Total computation time for that object was approximately 8 minutes. Its data structure was 3K in size whereas the commands used to

Fig. 8. Primitive objects.



build it, themselves an alternative description, accupied 300 words. The program is fully described in [2].

4. Conclusion

The main result of this work is that objects bounded by many mathematically simple faces are easily built up by adding and subtracting simpler objects. Efficient algorithms can be devised to carry out addition and subtracting if the operations are restricted in certain respects. Even with these restrictions and surfaces limited to planes and parts of cylinders, the variety of objects which can be made covers a good proportion of the machined components found in practice. The simple command language used to drive the program seems adequate for engineering designers, though an algorithmic language might also be employed. The latter approach has been followed in the EUKLID system [3].

Acknowledgments. The author wishes to thank C.A. Lang and A.R. Grayer of the CAD Group, Computer Laboratory, Cambridge, whose interest and suggestions have greatly benefited this work.

Received February 1974; revised July 1974

References

1. Bellman, R., et al. Abstraction and pattern classification. *J. Math. Anal. and Applications 13* (1966), 1–7.
2. Braid, I.C. *Designing with Volumes* (2nd ed.). Cantab Press, 97 Hurst Park Ave, Cambridge, U.K., 1974.
3. Engeli, M., and Hrdliczka, V. *EUKLID-eine Einfiihrung.* Fides Rechenzentrum, Zurich, 1974.
4. Lang, C.A. SAL—systems assembly language. Proc. AFIPS 1969 SJCC. Vol. 38, pp. 543–555, AFIPS Press, Montvale, N.J.
5. Newman, W.M., and Sproull, R.F. *Principles of Interactive Computer Graphics.* McGraw-Hill, New York, 1973.

Programming Techniques

G. Manacher Editor

The Quadratic Hash Method When the Table Size Is Not a Prime Number

Vladimir Batagelj
"Jožef Stefan" Institute,
University of Ljubljana

Previous work on quadratic hash methods is limited mainly to the case where the table size is a prime number. Here, certain results are derived for composite numbers. It is shown that all composite numbers containing at least the square of one of the component primes have full-period integer-coefficient quadratic hash functions.

Key Words and Phrases: quadratic search, hash code, scatter storage, table size

CR Categories: 3.74, 4.10

From the literature one gains the impression that the period of quadratic search is usually too small for effective use when the table size is not a prime number [1]. For this reason, most authors limit themselves to quadratic methods for tables whose size is a prime number or a special prime power. For example, the sequence $z_i \equiv z_o + Ri + \frac{1}{2}i(i+1) \pmod{2^k}$ due to Hopgood and Davenport [2] has the period of search $2^k - R$. The coefficient $\frac{1}{2}$ is not an integer, which underlies the belief that integer-coefficient full-period quadratic hash functions may be rare or nonexistent. However, this belief, as we shall now show, is false.

Consider the sequence

$$z_i \equiv z_0 + ai + bi^2 \pmod{d} \qquad (1)$$

for $i = 0, 1, 2, \ldots$ where a and b are integer constants and d is the table size. We reduce the problem of the period of search to the question whether there exist in-

Copyright © 1975, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted provided that ACM's copyright notice is given and that reference is made to the publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

Author's address: "Jožef Stefan" Institute, Jamova 39, 61 001, Ljubljana p.p. 199, Yugoslavia.